

SERVICE ACTUEL · MIGRATION-ORCHESTRATOR

Format et validation des migrations transverses

5 capacités analysées

services/migration-orchestrator · Python, Pydantic, PyYAML et CLI cleyrop-migrate-lint. Aucun exécuteur, API HTTP, schéma de runs, Dockerfile ou chart dans l'état documenté et le périmètre de fichiers lu.

Définir et vérifier un corpus de migrations de cluster hors schémas propres aux services. Le dépôt contient des outils de validation utilisables ; il ne livre pas encore de service d'exécution.

Lecture de code**Pas de qualification runtime nouvelle**

Domaines cible : Migration, Audit, Maintenance, IAM.

Périmètre de lecture

- README, pyproject, corpus/schema.py, loader.py, targeting.py et lint/manifest.py lus.
- Absence de migration livrée indiquée par le composant ; aucune commande d'application exécutée.

Objets et cycle de vie

- Migration possède nom/version/ciblage et liste d'étapes ordonnées ; Step décrit precheck/apply/verify.
- LoadedMigration conserve le dossier source ; CorpusError et LintIssue localisent une erreur. Aucun objet Run durable n'est constaté.

Dépendances du service

- Pydantic et PyYAML pour les outils présents.
- Les exécuteurs SQL/Kubernetes/shell sont seulement des valeurs du schéma et une cible de conception, pas des adaptateurs exécutables livrés.

Fonctionnalités et compatibilité à préserver

MO-01 **Cible non qualifiée**

Valider la structure d'un manifeste de migration

COMPORTEMENT OBSERVÉ

Le modèle impose apiVersion cleyrop.tech/v1, kind Migration, un nom non vide, une version entière positive et au moins une étape. Chaque étape possède nom, executor sql/kubectl/shell, precheck et verify non vides ; apply est optionnel. Les noms d'étape doivent être uniques et les champs supplémentaires sont refusés. Cette validation décrit un contrat ; elle n'exécute ni SQL ni commande système.

UTILISATEURS ET RÔLES

Auteurs de migrations et relecteurs.

INTERFACES DANS LE CODE

corpus.schema.Migration ; Step ; Targets ; Migration.model_validate.

DONNÉES ET ÉTATS

Manifestes YAML et objets Pydantic en mémoire ; aucun run persistant.

DÉPENDANCES ET ÉCHANGES

Pydantic et format YAML du corpus.

COMPOTEMENT CIBLE PROPOSÉ

Migration Rust porte le même contrat versionné avec actions typées, droits et empreinte du manifeste. Le runner futur distingue observation, application et vérification, sans remplacer les migrations de schéma possédées par chaque service.

MIGRATION DE CETTE CAPACITÉ

Porter les fixtures valides et invalides vers le validateur Rust. Conserver les champs historiques ou définir une conversion explicite ; refuser les commandes non couvertes par le contrat.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un manifeste sans verify, avec étape dupliquée ou champ inconnu est refusé avant toute action. Une validation réussie ne crée aucun effet externe.

Références de l'analyse

[monorepo · services/migration-orchestrator/src/migration_orchestrator/corpus/schema.py:57](#) Contrat des étapes et blocs requis.

[monorepo · services/migration-orchestrator/src/migration_orchestrator/corpus/schema.py:112](#) Version, types et unicité des étapes.

MO-02 **Cible non qualifiée****Charger un corpus dans un ordre déterministe et refuser les doublons****COMPOTEMENT OBSERVÉ**

Le chargeur parcourt les sous-répertoires dans l'ordre lexicographique, ignore fichiers racine et dossiers commençant par point ou underscore, et exige manifest.yaml dans les autres dossiers. Il utilise un chargement YAML sûr, exige un mapping racine puis valide le modèle. Les erreurs conservent le chemin source. load garde le répertoire d'origine de chaque migration et refuse deux migrations portant le même nom, même dans deux dossiers distincts.

UTILISATEURS ET RÔLES

Auteurs du corpus et opérateurs.

INTERFACES DANS LE CODE

corpus.loader.discover ; load_manifest ; load ; CorpusError ; LoadedMigration.

DONNÉES ET ÉTATS

Liste ordonnée de manifestes validés et chemins de ressources relatives.

DÉPENDANCES ET ÉCHANGES

Système de fichiers du corpus et validateur de schéma.

COMPOTEMENT CIBLE PROPOSÉ

Le chargeur Rust fige un corpus et son digest avant exécution. Le runner persiste nom, version, hash, ordre et références de fichiers nécessaires pour qu'une reprise ne lise pas un corpus modifié silencieusement.

MIGRATION DE CETTE CAPACITÉ

Conserver l'ordre des préfixes et les erreurs localisées. Ajouter une politique de chemin pour les ressources de commande avant de permettre un exécuteur sql_file.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un dossier sans manifeste ou un nom dupliqué bloque le chargement. Une reprise avec le même nom/version mais un contenu différent est refusée tant qu'un changement de plan n'est pas approuvé.

Références de l'analyse

[monorepo](#) · [services/migration-orchestrator/src/migration_orchestrator/corpus/loader.py:64](#) Découverte déterministe et dossiers ignorés.

[monorepo](#) · [services/migration-orchestrator/src/migration_orchestrator/corpus/loader.py:131](#) Rejet des noms de migration dupliqués.

MO-03 **Cible non qualifiée****Déterminer si une migration cible le cluster****COMPORTEMENT OBSERVÉ**

targets.clusters comporte include et exclude. La valeur all dans include cible tous les clusters ; exclude prend toujours le dessus. Une liste include vide ne cible rien. Le matching est une comparaison des noms fournis, sans découverte de labels Kubernetes. La fonction retourne uniquement un booléen ; aucun saut ou run n'est encore persisté.

UTILISATEURS ET RÔLES

Exploitation multi-clusters.

INTERFACES DANS LE CODE

corpus.targeting.applies_to ; corpus.schema.Targets.

DONNÉES ET ÉTATS

Nom de cluster et listes de ciblage du manifeste.

DÉPENDANCES ET ÉCHANGES

Nom de cluster fourni par la configuration de l'outil.

COMPORTEMENT CIBLE PROPOSÉ

Migration Rust fige la décision de ciblage avec identité stable du cluster, version de plan et motif appliqué/ignoré. Une migration ne doit pas toucher un cluster hors scope même si son nom ou sa configuration dérive en cours de run.

MIGRATION DE CETTE CAPACITÉ

Reprendre les tests de la matrice include/all/exclude. Définir comment les noms historiques correspondent à une identité de cluster avant adoption du runner.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un cluster présent à la fois dans include et exclude est ignoré. include vide ne lance rien. La preuve de ciblage reste attachée au run repris.

Références de l'analyse

[monorepo](#) · [services/migration-orchestrator/src/migration_orchestrator/corpus/targeting.py:27](#) Priorité de exclude et sémantique de all.

MO-04 **Cible non qualifiée****Détecter les incohérences entre exécuteur et blocs d'action****COMPORTEMENT OBSERVÉ**

Le linter complète le schéma par des contrôles sémantiques. Il vérifie que precheck, apply et verify ne contiennent que les clés admises et que chaque bloc contient une entrée adaptée à executor. `sql_file` est admis à la place de `sql` uniquement pour apply. `target_db` vide est refusé pour SQL ; `target_db` est interdit aux autres types. Un ciblage dont include et exclude sont tous deux vides est signalé. Les contrôles portent la structure, pas la sûreté ni l'idempotence du SQL ou du shell.

UTILISATEURS ET RÔLES

Auteurs et reviewers de manifests.

INTERFACES DANS LE CODE

`lint.manifest._lint_migration` ; `_lint_step` ; `_lint_block` ; `LintIssue`.

DONNÉES ET ÉTATS

Liste de problèmes localisés par manifeste et étape.

DÉPENDANCES ET ÉCHANGES

Schéma partagé et chargeur YAML.

COMPORTEMENT CIBLE PROPOSÉ

Un validateur Rust conserve ces règles puis ajoute politique d'exécution, analyse des chemins, périmètre des credentials et préconditions de schéma. Le résultat lint reste distinct d'une preuve de migration réussie.

MIGRATION DE CETTE CAPACITÉ

Porter les diagnostics et leurs fixtures. Fermer l'écart du linter global qui charge chaque manifeste individuellement et ne passe pas par la vérification des noms dupliqués de load.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un bloc kubectl sous executor sql est refusé. `sql_file` dans verify est refusé. Deux noms identiques sont aussi détectés par la commande lint finale, au même titre que par le chargeur.

Références de l'analyse

[monorepo · services/migration-orchestrator/src/migration_orchestrator/lint/manifest.py:79](#) Cohérence `target_db` et blocs.

[monorepo · services/migration-orchestrator/src/migration_orchestrator/lint/manifest.py:138](#) Clés autorisées et correspondance avec executor.

[monorepo · services/migration-orchestrator/src/migration_orchestrator/lint/manifest.py:36](#) Parcours individuel qui ne vérifie pas les noms au niveau corpus.

MO-05 **Cible non qualifiée****Fournir une commande de validation avec codes de sortie exploitables****COMPORTEMENT OBSERVÉ**

Le paquet expose cleyrop-migrate-lint et un point d'entrée python `-m migration_orchestrator.lint`. La commande reçoit un répertoire, collecte les problèmes, les écrit sur stderr avec chemin et étape puis retourne 1. Si aucun problème n'est trouvé, elle annonce le nombre de migrations et retourne 0 ; un corpus vide peut donc réussir. Aucun cleyrop-migrate apply, API, worker d'exécution ou table de runs n'est livré dans le code lu. Le README précise aussi l'absence d'intégration CI/hook à cette étape.

UTILISATEURS ET RÔLES

Développeurs, CI et futurs opérateurs de migration.

INTERFACES DANS LE CODE

```
cleyrop-migrate-lint <corpus_path> ; python -m migration_orchestrator.lint <corpus_path> ;
lint.manifest.main.
```

DONNÉES ET ÉTATS

Sortie de commande et diagnostics ; pas d'historique d'exécution durable.

DÉPENDANCES ET ÉCHANGES

Packaging Python actuel ; futur runner Rust et API Kubernetes/SQL.

COMPORTEMENT CIBLE PROPOSÉ

La CLI Rust sépare lint, plan, apply, inspect et retry. Un runner futur prend un verrou et une époque, exécute des runs bornés, conserve preuves precheck/apply/verify et rend un reçu durable. L'autorité des commandes de migration reste PostgreSQL ; elle ne remplace pas les checkpoints publiés des datasets.

MIGRATION DE CETTE CAPACITÉ

Livrer d'abord le validateur compatible et son branchement CI. Construire ensuite un runner avec tests de crash et restauration avant toute première migration transverse réelle ; ne pas présenter les commandes futures comme existantes.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une validation invalide retourne non-zéro avec un chemin exploitable. Un corpus vide est rendu explicitement. Le futur apply interrompu entre action et verify reprend par precheck sans doubler l'effet.

Références de l'analyse

[monorepo · services/migration-orchestrator/src/migration_orchestrator/lint/manifest.py:183](#) Contrat CLI et codes de sortie.

[monorepo · services/migration-orchestrator/src/migration_orchestrator/lint/__main__.py:6](#) Point d'entrée module.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Corpus immuable identifié par hash, plans ciblés, approbations concrètes, états durables et preuves d'application.
- Les migrations de schéma locales restent détenues par leur service et suivent Expand/Migrate/Contract.

Plan de données proposé

- Aucun plan d'exécution livré aujourd'hui. La cible lance des jobs bornés avec credentials propres au cluster et à l'action, puis vérifie le résultat.

Découpage du core Rust

- Validateur et chargeur compatibles ; planificateur de ciblage ; runner et magasin de reçus à construire.

- Adaptateurs SQL et Kubernetes étroitement typés ; shell uniquement dans un périmètre déclaré et auditable.

Identité, autorisation et isolation

- Un manifeste validé n'est pas une autorisation d'exécuter son contenu.
- Le runner doit borner namespace, base et permissions, conserver l'acteur et ne pas inclure de credentials dans ses manifestes ou rapports.

Exploitation et reprise

- Le composant actuel est une bibliothèque/CLI de validation, sans service à déployer ni corpus de migration rempli.
- L'exploitation cible exige exclusion concurrente, reprise, délais, observabilité et restauration avant adoption d'une première action réelle.

Séquence de migration

- Porter les contrats et fixtures de validation sans créer de faux parcours d'application.
- Brancher lint sur la CI et vérifier noms uniques au niveau corpus.
- Construire stockage de runs, verrou/époque et exécuteurs ; qualifier une migration canari réexécutable.
- Ajouter déploiement et intégration des mises à niveau seulement après preuve de reprise.

Écarts et décisions encore ouverts

- Le runner, ses credentials, la table de runs et l'API ne sont pas implémentés dans le périmètre constaté.
- La politique de résolution de sql_file et d'immutabilité du corpus doit précéder son exécution.
- Le commentaire du linter suggère un include vide explicite pour une migration garée, mais le code signale tout ciblage où include et exclude sont vides ; ce comportement doit être clarifié.
- Aucun lint ni test du composant n'a été exécuté dans ce lot.

Références de l'analyse

[monorepo](#) · [services/migration-orchestrator/README.md:21](#) État actuel, outils présents et absence du runner/déploiement.

[monorepo](#) · [services/migration-orchestrator/pyproject.toml:16](#) Unique commande livrée cleyrop-migrate-lint.